

System zarządzania apteka

Projekt bazy danych +
aplikacja
(PostgreSQL • C# • Avalonia
UI)



Politechnika
Wrocławska

Cel i zakres projektu

Cel

- Lokalny system bazodanowy wspierający codzienną pracę apteki w małym mieście
- Jedna spójna baza: recepty, leki gotowe, receptura, magazyn, zamówienia
- Praca offline – połączenie wyłącznie z lokalnym PostgreSQL

Technologie

- PostgreSQL (baza relacyjna)
- C# + Avalonia UI (aplikacja desktopowa)
- ODBC (Open Database Connectivity)

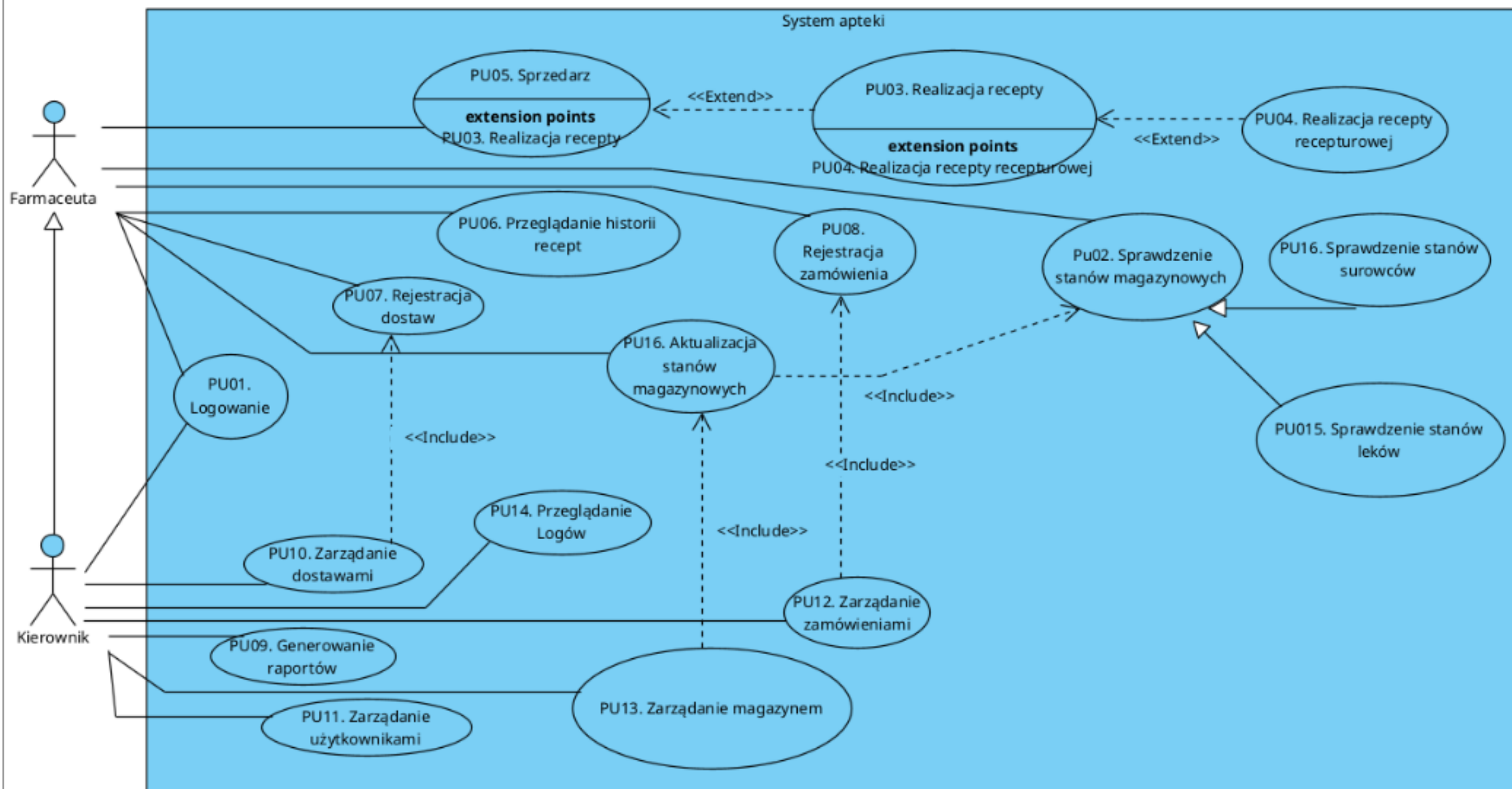
Użytkownicy i procesy

Kierownik apteki

Nadzór, zatwierdzanie zamówień, korekty i raporty, zarządzanie uprawnieniami.

Farmaceuta

Realizacja recept, przygotowanie leków robionych, dostawy, bieżące stany magazynowe.



Architektura Danych 3 schematy = 3 obszary domeny

apтека

dane biznesowe: osoby, klienci,
lekarze, recepty, sprzedaż, leki i
receptury

magazyn

gospodarka magazynowa:
dostawy, partie, surowce,
dostawcy, zamówienia

uzytkownicy

bezpieczeństwo: konta, role,
dziennik operacji (LogOperacji)

Dlaczego taki podział?

- Porządkuje projekt i upraszcza nadawanie uprawnień (GRANT/REVOKE)
- Oddziela dane wrażliwe (użytkownicy/logi) od danych operacyjnych
- Ułatwia rozwój: raporty i logika mogą rosnąć bez „bałaganu” w jednym schemacie

Spójność danych „z definicji”

Ograniczenia integralności

- PRIMARY KEY + sekwencje: jednoznaczne identyfikatory
- FOREIGN KEY: brak rekordów osieroconych (np. recepta bez klienta)
- UNIQUE: PESEL, PWZ, EAN, login – brak duplikatów
- CHECK/NOT NULL: walidacja domen (statusy, typy, daty, ilości)

Indeksy (wydajność)

- Automatycznie: indeksy na PK i UNIQUE
- Dodatkowo: idx_leki_nazwa, idx_surowce_nazwa
- Dla raportów: idx_recepta_data_realizacji
- Efekt: szybkie wyszukiwanie + filtrowanie po dacie

Tip: walidacja jest podwójna — część w aplikacji (formularze), a „ostatnie słowo” ma baza (constraints).

Logika po stronie bazy

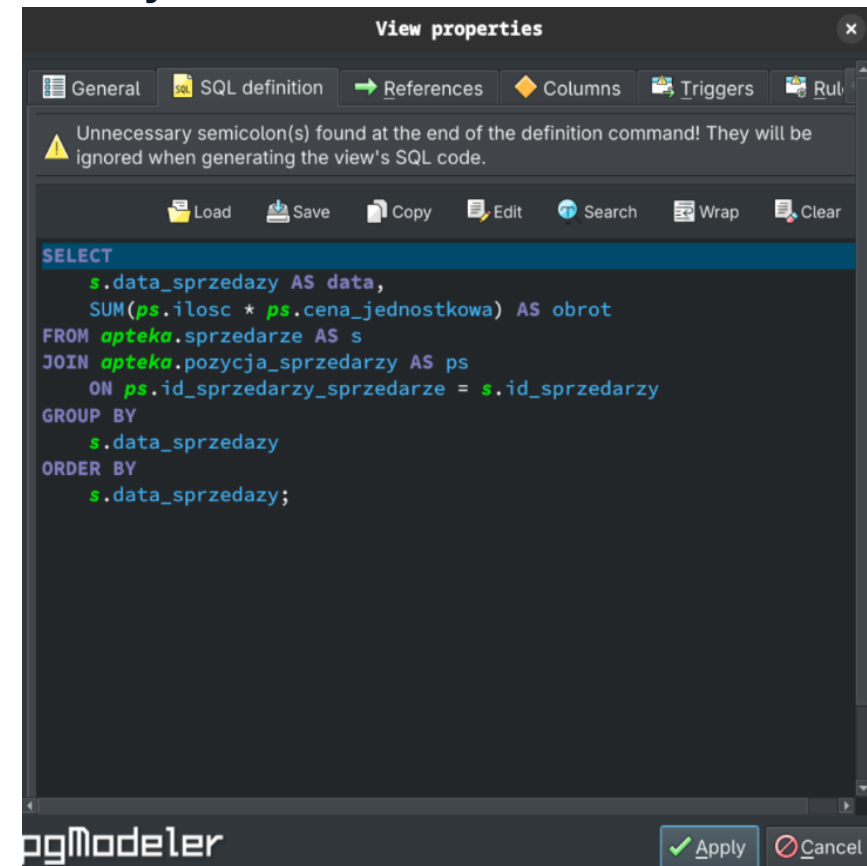
Widoki (VIEW)

- v_stan_magazynu_lekow: sumy ilości + najbliższa data ważności
- v_stan_magazynu_surowcow: analogicznie dla receptury
- v_sprzedaz_dzienna: obrót dzienny (ilość × cena)

Audyt (FUNCTION + TRIGGER)

- fn_zapisz_log_operacji zapisuje: kto/co/kiedy/rekord
- Triggery na INSERT/UPDATE/DELETE (np. recepta, sprzedaż, dostawy)
- Id użytkownika z sesji: SET app.user_id

Przykładowy widok



Role + GRANT + log operacji

Założenia

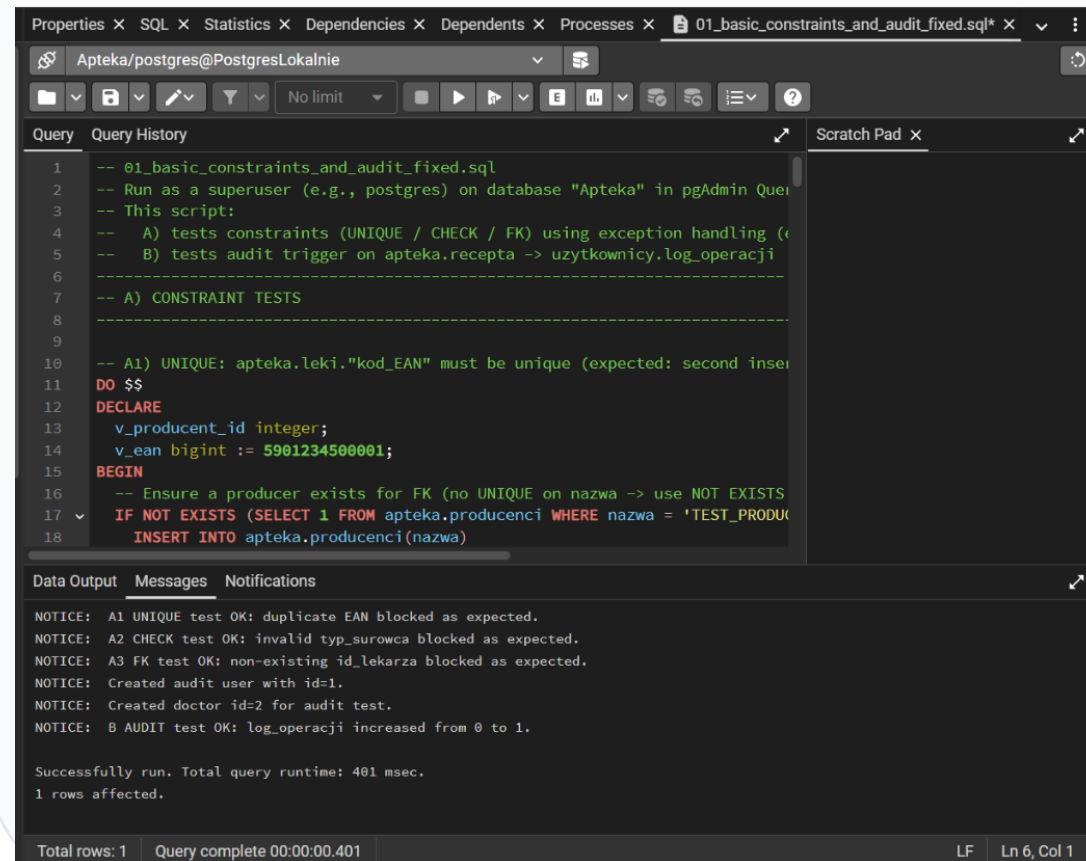
- rola_farmaceuta: praca operacyjna (SELECT/INSERT/UPDATE w wymaganych tabelach)
- rola_kierownik: rozszerzone CRUD + dostęp do logów operacji
- Zasada najmniejszych uprawnień (blokada nieautoryzowanych działań)
- Dane osobowe w osobnych tabelach → łatwiej ograniczać dostęp

Testy bazy danych (pgAdmin)

Testy poprawności ograniczeń (integralność danych)

- **UNIQUE**: próba wstawienia dwóch leków z tym samym kodem EAN (drugi INSERT odrzucony).
- **CHECK**: próba wstawienia surowca z niedozwolonym typem (odrzucone przez CHECK).
- **FOREIGN KEY**: próba wstawienia recepty wskazującej na nieistniejącego lekarza (odrzucone przez FK).

Test audytu (triggery i funkcje)



The screenshot shows the pgAdmin interface with a SQL query editor and its output. The query is a PL/pgSQL script for testing database constraints and audit triggers. The output shows several notices confirming that the tests passed as expected.

```
1 -- 01_basic_constraints_and_audit_fixed.sql
2 -- Run as a superuser (e.g., postgres) on database "Apteka" in pgAdmin Quer
3 -- This script:
4 --   A) tests constraints (UNIQUE / CHECK / FK) using exception handling (
5 --   B) tests audit trigger on apteka.recepta -> uzytkownicy.log_operacji
6 -----
7 -- A) CONSTRAINT TESTS
8 -----
9
10 -- A1) UNIQUE: apteka.leki."kod_EAN" must be unique (expected: second inser
11 DO $$
12 DECLARE
13     v_producent_id integer;
14     v_ean bigint := 5901234500001;
15 BEGIN
16     -- Ensure a producer exists for FK (no UNIQUE on nazwa -> use NOT EXISTS
17     IF NOT EXISTS (SELECT 1 FROM apteka.producenci WHERE nazwa = 'TEST_PRODUK
18     INSERT INTO apteka.producenci(nazwa)
```

Data Output Messages Notifications

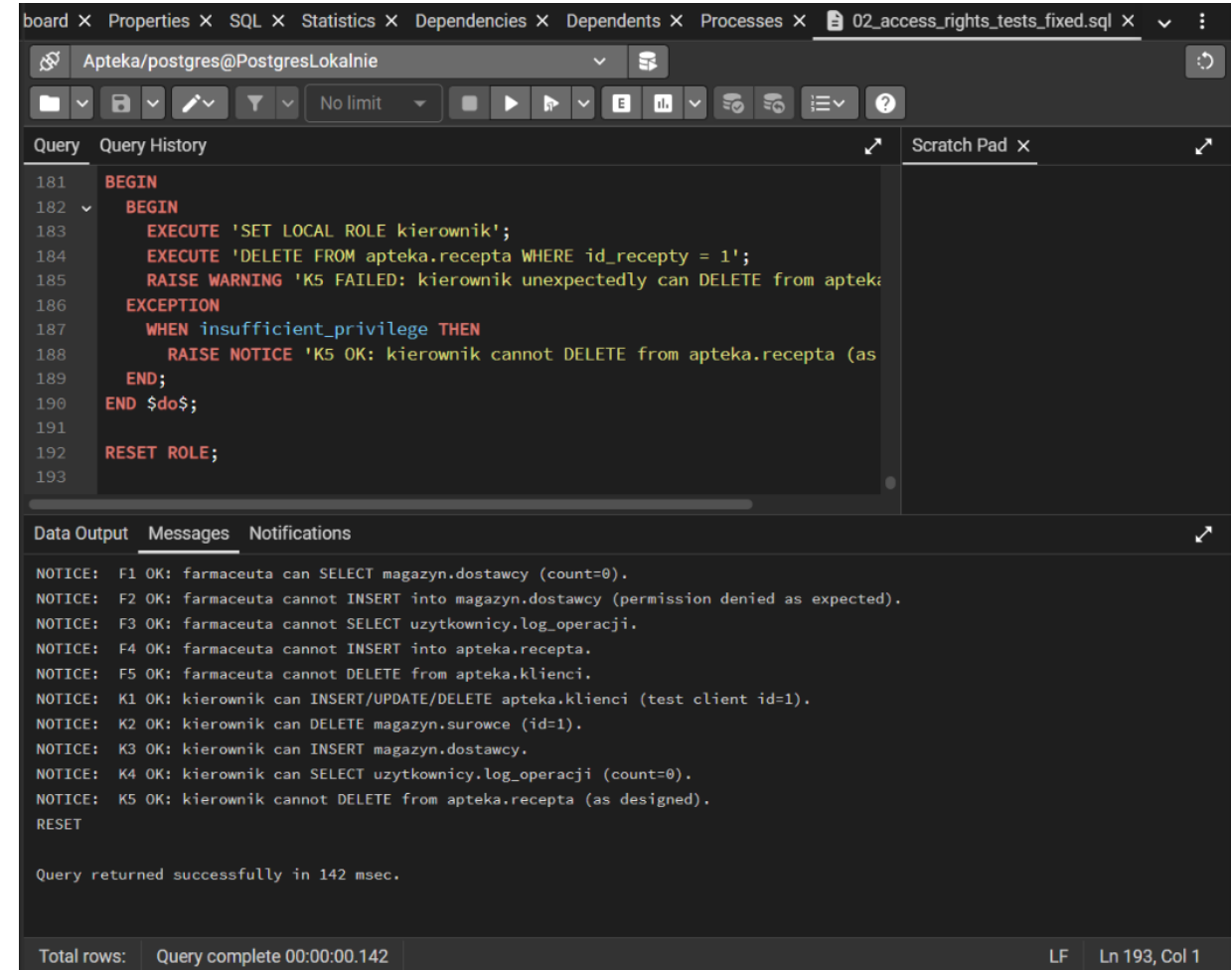
```
NOTICE: A1 UNIQUE test OK: duplicate EAN blocked as expected.
NOTICE: A2 CHECK test OK: invalid typ_surowca blocked as expected.
NOTICE: A3 FK test OK: non-existing id_lekarza blocked as expected.
NOTICE: Created audit user with id=1.
NOTICE: Created doctor id=2 for audit test.
NOTICE: B AUDIT test OK: log_operacji increased from 0 to 1.

Successfully run. Total query runtime: 401 msec.
1 rows affected.
```

Total rows: 1 Query complete 00:00:00.401 LF Ln 6, Col 1

Testy Uprawnień role + GRANT

- Testy przez logowanie jako farmaceuta i kierownik
- Oczekiwane: operacje zabronione blokowane komunikatem permission denied
- Farmaceuta: SELECT/INSERT/UPDATE w wybranych obszarach + blokada DELETE tam, gdzie brak prawa
- Kierownik: szerszy CRUD + dostęp do logów



The screenshot shows a PostgreSQL client window with the following components:

- Query Editor:** Contains a SQL script with line numbers 181 to 193. The script starts with `BEGIN`, followed by a block of code enclosed in `EXECUTE 'SET LOCAL ROLE kierownik';` and `END $do$;`. The code includes a `DELETE` statement on line 184, an `EXCEPTION` block on lines 186-188, and a `RESET ROLE;` statement on line 192.
- Data Output / Messages:** Displays the results of the query execution. It shows a series of `NOTICE` messages (F1 through K5) and a final `RESET` message. The messages indicate successful operations for the `farmaceuta` and `kierownik` roles, as well as expected permission denials.
- Status Bar:** At the bottom, it shows "Total rows: Query complete 00:00:00.142" and "Ln 193, Col 1".

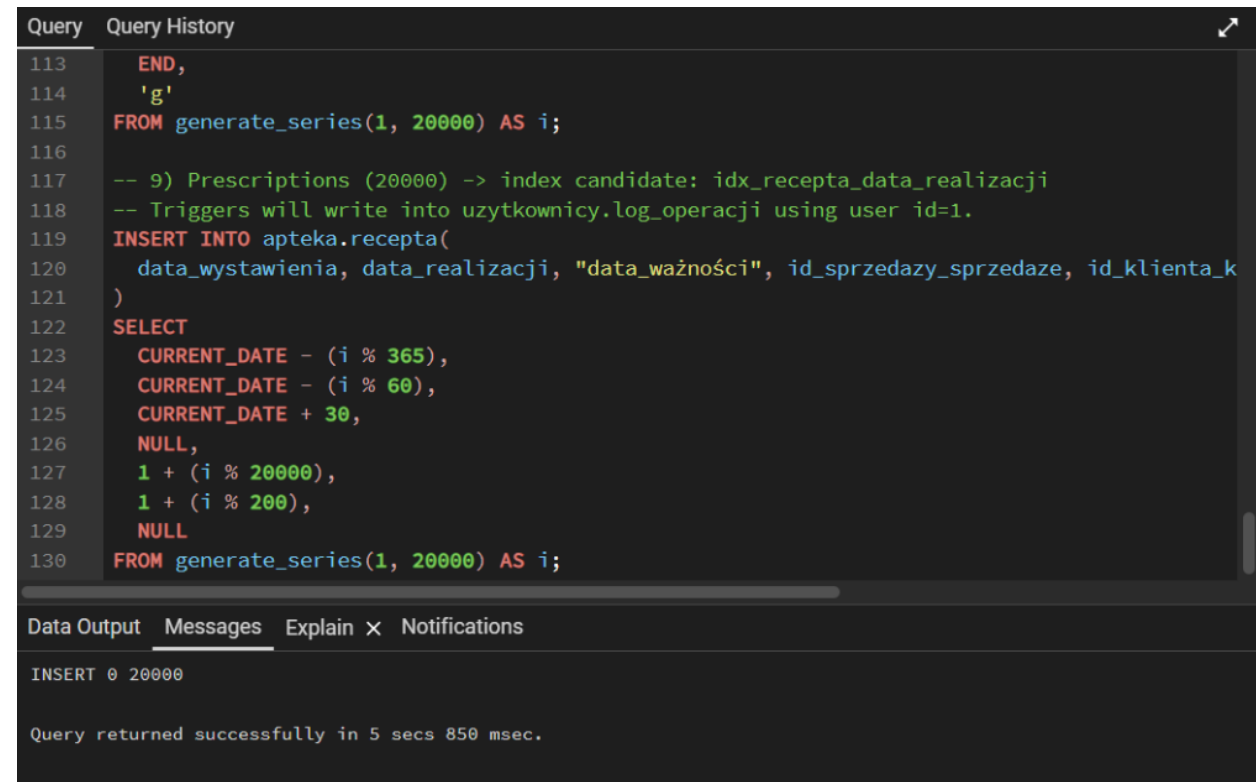
```
181 BEGIN
182 BEGIN
183 EXECUTE 'SET LOCAL ROLE kierownik';
184 EXECUTE 'DELETE FROM apteka.recepta WHERE id_recepty = 1';
185 RAISE WARNING 'K5 FAILED: kierownik unexpectedly can DELETE from apteka
186 EXCEPTION
187 WHEN insufficient_privilege THEN
188 RAISE NOTICE 'K5 OK: kierownik cannot DELETE from apteka.recepta (as
189 END;
190 END $do$;
191
192 RESET ROLE;
193
```

NOTICE: F1 OK: farmaceuta can SELECT magazyn.dostawcy (count=0).
NOTICE: F2 OK: farmaceuta cannot INSERT into magazyn.dostawcy (permission denied as expected).
NOTICE: F3 OK: farmaceuta cannot SELECT uzytkownicy.log_operacji.
NOTICE: F4 OK: farmaceuta cannot INSERT into apteka.recepta.
NOTICE: F5 OK: farmaceuta cannot DELETE from apteka.klienci.
NOTICE: K1 OK: kierownik can INSERT/UPDATE/DELETE apteka.klienci (test client id=1).
NOTICE: K2 OK: kierownik can DELETE magazyn.surowce (id=1).
NOTICE: K3 OK: kierownik can INSERT magazyn.dostawcy.
NOTICE: K4 OK: kierownik can SELECT uzytkownicy.log_operacji (count=0).
NOTICE: K5 OK: kierownik cannot DELETE from apteka.recepta (as designed).
RESET

Query returned successfully in 142 msec.

Testy wydajności

- Dane: generowane automatycznie generate_series(1, N) dla 10 000 i 20 000 rekordów (porównywalne serie)
- Po seedzie: VACUUM (ANALYZE), żeby planner miał aktualne statystyki
- Pomiar: EXPLAIN (ANALYZE, BUFFERS) → zapis planu (Index Scan vs Seq Scan) i czasu wykonania
- Porównanie wariantów: z indeksami i po ich czasowym usunięciu (DROP INDEX)
- Indeksy użyte w testach porównawczych:
idx_leki_nazwa, idx_surowce_nazwa,
idx_recepta_data_realizacji



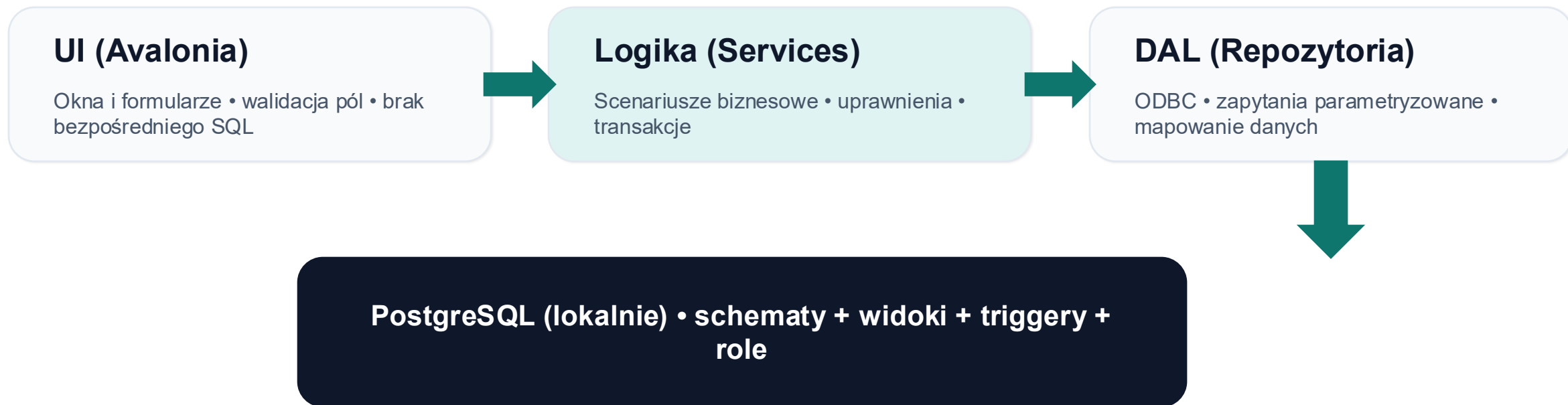
```
113      END,  
114      'g'  
115  FROM generate_series(1, 20000) AS i;  
116  
117  -- 9) Prescriptions (20000) -> index candidate: idx_recepta_data_realizacji  
118  -- Triggers will write into uzytkownicy.log_operacji using user id=1.  
119  INSERT INTO apteka.recepta(  
120      data_wystawienia, data_realizacji, "data_ważności", id_sprzedazy_sprzedaze, id_klienta_k  
121  )  
122  SELECT  
123      CURRENT_DATE - (i % 365),  
124      CURRENT_DATE - (i % 60),  
125      CURRENT_DATE + 30,  
126      NULL,  
127      1 + (i % 20000),  
128      1 + (i % 200),  
129      NULL  
130  FROM generate_series(1, 20000) AS i;
```

Data Output Messages Explain × Notifications

INSERT 0 20000

Query returned successfully in 5 secs 850 msec.

Trójwarstwowy podział odpowiedzialności



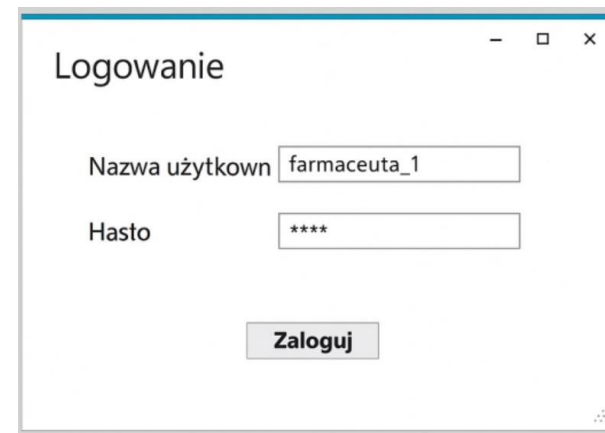
- UI komunikuje się tylko z logiką (brak „SQL w widokach”)
- Parametryzacja zapytań + transakcje dla operacji wieloetapowych
- Po logowaniu ustawiamy `app.user_id`, żeby audyt znał realnego użytkownika

Prosty UI pod pracę w aptece

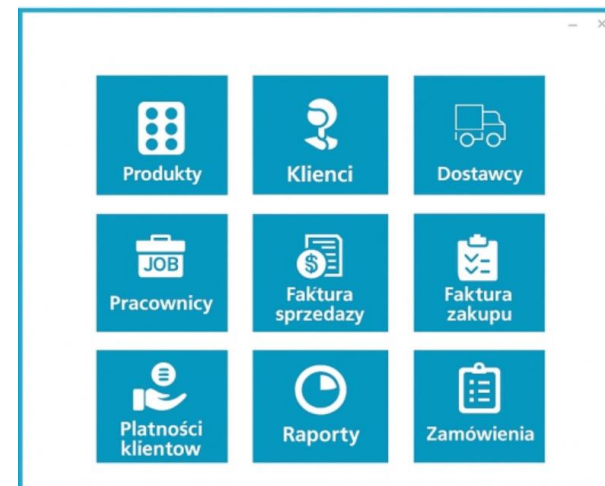
Założenia UX

- Menu kafelkowe: szybkie wejście do modułów
- W modułach: lista + wyszukiwanie + CRUD
- Widoczność funkcji zależna od roli
- Walidacja danych w formularzach

Wizualizacja UI



A screenshot of a web application window titled "Logowanie". It contains two input fields: "Nazwa użytkownika" with the value "farmaceuta_1" and "Hasło" with masked characters "****". Below the fields is a button labeled "Zaloguj".



Aplikacja desktopowa

Opis aplikacji

- Desktopowa aplikacja dla apteki (Avalonia UI, C#)
Służy do codziennej pracy: szybka nawigacja po modułach (menu kafelkowe) i obsługa danych operacyjnych apteki.
- Modułowa struktura ekranów + stały wzorzec pracy
- Kontrola dostępu i zachowanie zależne od roli
Po zalogowaniu aplikacja uwzględnia rolę (np. farmaceuta/kierownik)

Rzeczywiste UI

Logowanie

Nazwa użytkownika:

Hasło:



Szuka:

Filtruj

Wyczyść

ID	Nazwa	Producent	Substancja Czynna	Bez Recepty
1	Apap	Polpharma S.A.	Paracetamol	☒
2	Ibuprom	Polpharma S.A.	Ibuprofen	☒
3	Duomox	Bayer Sp. z o.o.	Amoxicillinum	☐

[Dodaj](#) [Edytuj](#) [Usuń](#)

ID:	1	EAN	Dawka	Ilość	Dostępność	Postać
Nazwa:	Apap	5900000000001	500 mg	12	109	Kapsulka
Producent:	Polpharma S.A.	5900000000002	500 mg	24	0	Kapsulka
Adres:	Poliska, Fabryczna, 100/A, Łódź, 90-000					
Bez Recepty:	☐					
Substancja Czynna:	Paracetamol					

[Zapisz](#) [Anuluj](#)

Od:

1

styczeń

0001

Do:

16

styczeń

2032

Sprzedaż

Magazyn

Typ	Nazwa	Ilość	Data Ważności
	Apap 500 mg x 12	109	2026-04-25
	Duomox 1000 mg x 20	19	2027-01-01
	Ibuprom 200 mg x 10	49	2026-06-30
	Mentol	1000	2026-12-31
	Wazelina biała	5000	2028-01-01



Adresy

ID:

0

Kraj:

Niemcy

Ulica:

Numer Domu:

12

Numer Lokalu:

Miejscowość:

Görlitz

Kod Pocztowy:

|

Pole "KodPocztowy" nie może być puste

Zapisz

Anuluj

Zamknij



Projekt przygotowali: Kacper Wajda, Robert Tworek, Michał Gładkojć